

Floating Point 行為專案報告

Advanced C++ Homework 2

學號：113062331

姓名：林欣磊

繳交日期：2026/10/07

Q0. 實驗環境

項目	主要平台	輔助平台
裝置	ASUS TUF Gaming F16	MacBook Air (2025)
CPU／架構	Intel Core i7-14650HX／x86-64	Apple M4／arm64
系統	Windows 11／WSL2 Ubuntu 26.04.1 LTS	macOS 27.0.1
編譯器	GNU GCC 16.2.0 (本地建置)	GNU GCC 16.2.0 (Homebrew)
數學函式庫	glibc 2.43-2ubuntu2.4	macOS 系統函式庫

主要實驗於 WSL2 Ubuntu 執行，Mac 僅用於 Q5 的跨平台比較。Q1 的 Benchmark 未固定 CPU 頻率與溫度，因此數值用來比較本次測試條件下各型態與運算方式的相對效能，不視為硬體的絕對效能。

實驗程式以 `sizeof` 與 `std::numeric_limits<T>::digits` 分別記錄各型態的儲存空間與二進位有效位數。WSL 的 long double 採用 80-bit extended precision，但依 ABI 對齊後實際占用 16 bytes；16 bytes 是儲存大小，不代表具有 128-bit 的運算精度。

平台／型態	儲存空間 (bytes)	有效位數 (digits)
WSL float	4	24
WSL double	8	53
WSL long double	16	64
Mac long double	8	53

後續實驗以 float、double 與 long double 為主要比較對象。除 Q3 的 fast-math 對照與 Q4 的捨入模式測試外，皆使用預設的最接近值捨入並關閉 fast-math。這些型態差異會在 Q1 的精度測試與 Q5 的跨平台結果中再次出現。

Q1. Benchmark

速度比較

速度測試讓三種浮點型態執行相同形式的加減、乘法與除法。「連續計算」使每次運算依賴前一次結果，用來觀察資料相依鏈下的成本；「四組並行」則維持四組互不依賴的值，讓處理器有機會重疊執行運算。每組先暖機 2 次，再正式計時 7 次，每次執行 5,000,000 回合，最後取中位數。每回合的加減、乘法與除法皆採用成對的互逆操作，例如 `+step/-step`、`×factor/×inverse` 與 `/factor`、`/inverse`，使數值在大量迭代後仍維持於相近尺度，避免數值持續放大、縮小、溢位或進入極小值範圍而干擾效能比較。

```
// Continuous dependency chain
for (std::size_t i = 0; i < n; ++i) { a /= factor; a /= inverse; }

// Four independent chains
for (std::size_t i = 0; i < n; ++i) {
    a /= factor; b /= factor; c /= factor; d /= factor;
    a /= inverse; b /= inverse; c /= inverse; d /= inverse;
}
```

圖 1-1 連續相依鏈與四組獨立運算鏈（以除法為例）。

連續計算與四組並行每回合分別執行 2 次與 8 次算術運算，總耗時除以實際運算次數後得到 ns/op，數值越小代表本測試中的平均運算成本越低。程式以 GCC 16.2.0、-O3 編譯，並關閉 fast-math、FMA、LTO 與自動向量化。為避免編譯器移除待測運算，程式保留並檢查最終結果，也確認產生的機器碼中仍存在算術迴圈。由於計時仍包含迴圈控制與函式呼叫等成本，這裡的 ns/op 不等同於單一 CPU 指令的 latency。

下表為 7 次計時的中位數，單位 ns/op。

運算	模式	float	double	long double
加減	連續計算	0.432182	0.429384	0.645650
加減	四組並行	0.117829	0.117475	0.275822
乘法	連續計算	0.860375	0.858836	0.859048
乘法	四組並行	0.215455	0.215547	0.216369
除法	連續計算	2.333131	2.999956	3.326920
除法	四組並行	0.617730	0.828075	0.923359

連續乘法中，三種型態皆約為 0.86 ns/op，差異不明顯；連續除法則分別為 2.333、3.000、3.327 ns/op，其中 float 最快。改成四組互不相依的運算後，除法降至 0.618、0.828、0.923 ns/op。這表示效能不只與型態有關，資料相依性也會限制處理器同時進行多筆運算的能力。

產生的機器碼顯示，float 與 double 使用 SSE 指令，而 long double 使用 x87 指令。因此，浮點型態的速度不能只由 sizeof 推測，還要考慮實際指令與處理器的執行方式。[4]

大數加總的精度差異

速度之外，有限精度也會直接影響計算結果。為觀察運算順序造成的差異，我將 [大數, 1, -大數] 重複 10,000 組，再分別正序與反序累加；大數取 2^{24} 與 2^{53} ，兩組資料在精確算術下的答案都應為 10,000。

```
for (std::size_t i = 0; i < blocks; ++i) {
    input.push_back(big);
    input.push_back(T(1));
    input.push_back(-big);
}
for (T value : input) forward += value;
for (auto i = input.rbegin(); i != input.rend(); ++i) reverse += *i;
```

圖1-2 大數加總實驗：建立相同輸入後分別以正序與反序累加。

型態	大數	正序結果	反序結果	絕對誤差 正／反
float	2^{24}	0	10000	10000／0
double	2^{24}	10000	10000	0／0
long double	2^{24}	10000	10000	0／0
float	2^{53}	0	0	10000／10000
double	2^{53}	0	10000	10000／0
long double	2^{53}	10000	10000	0／0

float 的格式包含 1 位符號、8 位指數與 23 位 fraction；double 則為 1／11／52 位。正規值還有隱含的最高有效位，因此兩者的二進位有效位數分別為 24 與 53。WSL 的 x87 80-bit extended precision 則有 64 位有效數字。浮點加減必須先對齊指數，再把結果捨入到型態能保存的精度，因此大數附近過小的增量可能在捨入時消失。[3][4]

結果中，float 在 2^{24} 的正序加總得到 0，反序則得到 10,000；double 在 2^{53} 時也出現相同現象。WSL 的 long double 有 64 位有效位數，因此兩組測試都得到 10,000。當大數提高到 2^{53} 時，float 已無法保留這個尺度附近的 1，正序與反序都產生誤差。這組實驗說明，即使資料型態不變，運算順序仍可能改變最終答案。

Q2. Library Implementation

本題選擇 `<cmath>` 的 `std::sqrt` 作為分析對象。C++20 提供 float、double 與 long double 的平方根運算，整數輸入則可轉為 double 處理；對負有限值的呼叫屬於定義域錯誤，錯誤回報與 `math_errhandling` 有關。標準規範的是函式的可觀察行為，並不要求一定呼叫 libm，也不指定必須使用某一條 CPU 指令。[1]

測試方法

我建立兩個使用相同測試資料的版本。一般版本直接呼叫 `std::sqrt`，讓編譯器依正常流程產生程式碼；另一版本透過 `volatile` 函式指標呼叫 `sqrtf`/`sqrt`/`sqrtl` 的函式庫符號。測試涵蓋一般值、負值、 ± 0 、無窮大、NaN、最小 `normal` 與最小 `subnormal`，並在每次呼叫前清除 `errno` 與浮點例外旗標。除了比較兩個版本的輸出，我也檢查編譯後的 `assembly`，藉此區分介面規格、實際指令路徑與執行結果。

```
#ifdef HW2_FORCE_LIBM
__attribute__((noinline)) double sqrt_path(double x) {
    double (*volatile f)(double) = &::sqrt;
    return f(x);
}
#else
__attribute__((noinline)) double sqrt_path(double x) {
    return std::sqrt(x);
}
#endif
```

圖2-1 一般 `std::sqrt` 路徑與強制 `libm` 符號呼叫的實作。

```
std::feclearexcept(FE_ALL_EXCEPT);
errno = 0;
T result = sqrt_path(x);
int saved_errno = errno;
int flags = std::fetestexcept(FE_ALL_EXCEPT);
```

圖2-2 每次平方根運算前清除狀態，並在輸出前立即保存 `errno` 與浮點例外旗標。

實際指令路徑

反組譯結果顯示，GCC 16.2.0 在一般版本處理正常有限輸入時，`float`、`double` 與 `long double` 分別使用 `SQRTSS`、`SQRTSD` 與 `x87 FSQRT`；負數路徑則會轉交函式庫。強制經過 `libm` 符號後，本機 `glibc` 的有限值實作中仍可觀察到硬體平方根指令。也就是說，在這次 WSL x86-64 環境中，常見輸入的平方根主要由處理器提供的平方根指令完成。關鍵指令如下：

```
float : SQRTSS      double : SQRTSD      long double : FSQRT
libm 的 __sqrt_finite : sqrtsd %xmm0, %xmm0
libm 的 __sqrtl_finite : fsqrt
```

數值與特殊值驗證

接著比較兩個版本的可觀察結果。每次呼叫後立即保存 `errno` 與浮點例外旗標；30 組「型態 × 輸入」測試中，兩條路徑得到相同的數值與錯誤狀態。例如 `sqrt(4)` 精確得到 2；`sqrt(2)` 得到各型態可表示的近似值；`sqrt(-1)` 在本次環境得到 NaN、`errno=33` 並設置 `FE_INVALID`；`sqrt(-0)` 則保留負零。代表性結果整理如下。

型態	CPU 指令／負數處理	<code>sqrt(2)</code> 實測
float	<code>SQRTSS</code> / <code>sqrtf</code>	1.41421354
double	<code>SQRTSD</code> / <code>sqrt</code>	1.4142135623730951
long double	<code>FSQRT</code> / <code>sqrtl</code>	1.41421356237309504876

通用軟體實作

硬體平方根指令是本機觀察到的主要路徑，但不是 C++ 規格要求的唯一實作方式。作為軟體實作的參考，glibc 的 `generic double sqrt` 會先縮放指數範圍，把問題轉到較小區間，再估計 $1/\sqrt{s}$ ，透過誤差修正與多項式提高精度，最後恢復尺度並處理正確捨入；極小值還需要額外縮放。[5a] 這段 `generic source` 用來說明沒有直接採用上述本機路徑時可使用的數值方法，不代表本次正常輸入實際執行了這套流程。

因此，`std::sqrt` 在原始碼中雖然只是一次函式呼叫，實際執行方式仍取決於編譯器、函式庫與目標平台。本次結果以 `assembly` 確認本機主要使用的硬體指令，再用實際輸出驗證一般值與特殊值的行為，兩者與 C++ 規格分別對應到「如何實作」與「應呈現什麼行為」。

Q3. IEEE 754 與 fast-math

`-ffast-math` 會放寬編譯器必須保留的浮點語意，例如允許重新結合運算、假定不會出現 NaN 或無窮大，並放寬對負零、`errno` 與動態捨入模式的要求，以換取更多最佳化空間。由於 IEEE 754 運算的中間結果可能發生捨入，浮點加法不能像實數運算一樣任意改變括號。[2]

為隔離 `fast-math` 的影響，同一份程式與相同輸入只切換 `-fno-fast-math` / `-ffast-math`；兩者皆使用 GCC 16.2.0、`-O3`，並關閉 LTO 與 FMA。輸入設定為 `a=1e16`、`b=-1e16`、`c=1`，比較 `(a+b)+c` 與 `a+(b+c)`，並另外測試 NaN 的 `x!=x`。

```
double left_grouped(double a, double b, double c) {
    return (a + b) + c;
}
double right_grouped(double a, double b, double c) {
    return a + (b + c);
}
bool self_inequality(double x) {
    return x != x;
}
```

圖3-1 Q3 的三個待測操作：兩種加法分組與 NaN 自身不等比較。

觀察	關閉 fast-math	開啟 fast-math
<code>(a+b)+c</code>	1	0
<code>a+(b+c)</code>	0	0
NaN 的 <code>x!=x</code>	true	false
一般數值對照組	6	6

```

$alan@TUF-F16:/mnt/c/Users/ALAN/Desktop/floating-point-hw2$ cat results/linux-20261004-201349/q3-strict.txt
echo "===== FAST ====="
cat results/linux-20261004-201349/q3-fast.txt
$ build/run-linux-20261004-201349/q3_strict 1e16 -1e16 1
experiment=Q3 fast math
compiler=16.2.0
rounding_mode=0
fast_math=false
a=10000000000000000,hex=0x1.1c37937e08p+53
b=-10000000000000000,hex=-0x1.1c37937e08p+53
c=1,hex=0x1p+0
left_grouped=1,hex=0x1p+0
right_grouped=0,hex=0x0p+0
nan_input_bits=0x7ff8000000000001
nan_self_inequality=true
finite_control=6,hex=0x1.8p+2

[exit=0]
===== FAST =====
$ build/run-linux-20261004-201349/q3_fast 1e16 -1e16 1
experiment=Q3 fast math
compiler=16.2.0
rounding_mode=0
fast_math=true
a=10000000000000000,hex=0x1.1c37937e08p+53
b=-10000000000000000,hex=-0x1.1c37937e08p+53
c=1,hex=0x1p+0
left_grouped=0,hex=0x0p+0
right_grouped=0,hex=0x0p+0
nan_input_bits=0x7ff8000000000001
nan_self_inequality=false
finite_control=6,hex=0x1.8p+2

[exit=0]

```

圖 3-2 相同輸入下，開啟 fast-math 後 left_grouped 由 1 變為 0，NaN 判斷由 true 變為 false。

關閉 fast-math 時， $(a+b)+c$ 先讓兩個大數抵消再加 1，因此得到 1； $a+(b+c)$ 則先計算 $-1e16+1$ ，這個增量因精度不足而消失，最後得到 0。開啟 fast-math 後，第一式也變成 0，assembly 顯示編譯器改成先計算 $b+c$ ；同時，NaN 的 $x!=x$ 也由 true 變成 false。這證明 fast-math 不只是效能選項，它允許編譯器採用會改變浮點結果與特殊值語意的最佳化。[2]

Q4. 浮點數輸出

若把「四捨五入」理解為遇到正中點就一律進位，這個描述並不精確。為避免十進位 literal 本身先產生表示誤差，我選擇能被 double 精確表示的 $1/1024 = 0.0009765625$ ；它正好位於兩個九位小數 0.000976562 與 0.000976563 的中點。實驗同時比較 `printf("%.9f")` 與 `std::fixed << std::setprecision(9)`，並測試中點兩側的相鄰浮點值、負值及四種捨入模式。

```

double tie = std::ldexp(1., -10); // 1/1024 = 0.0009765625
std::snprintf(printf_text, sizeof printf_text, "%.9f", tie);
stream << std::fixed << std::setprecision(9) << tie;

// Directly adjacent representable values are also tested.
std::nextafter(tie, 0.);
std::nextafter(tie, 1.);

```

圖 4-1 以可精確表示的中點及其相鄰浮點值比較兩種九位小數輸出。

模式	正中點：兩種輸出	負中點：兩種輸出
最接近值	0.000976562	-0.000976562
向上	0.000976563	-0.000976562
向下	0.000976562	-0.000976563
朝零	0.000976562	-0.000976562

在最接近值模式下，兩種輸出都得到 0.000976562，也就是第十位雖然是 5，第九位並沒有進位，已足以反駁「逢五必進」的說法。中點兩側的相鄰值分別輸出 0.000976562 與 0.000976563；四種捨入模式共 16 組比較中，printf 與 iostream 的結果一致。

本機在這個中點案例選擇末位為偶數的 0.000976562，而向上或向下捨入會得到不同結果。因此，更精確的說法是：這兩種寫法會依目前的捨入規則，把實際儲存的浮點值格式化到小數後九位，而不是對原始十進位數做固定的「逢五進一」。

Q5. 加分題

Q0 已經顯示 WSL 與 Mac 的 long double 精度不同。選擇 $(2^{53}+1)-2^{53}$ ，是因為 2^{53} 正好能放大兩個平台 long double 有效位數不同造成的差異：若中間結果能保留 +1，最後會得到 1；若 +1 在中間運算時被捨入掉，最後則會得到 0。為確認這個差異是否真的會改變計算結果，我使用相同的 long double 算式 $(2^{53}+1)-2^{53}$ 進行跨平台測試。結果在 WSL x86-64 為 1，在 Apple M4 arm64 為 0，而精確數學答案為 1。兩台皆使用 GNU GCC release 16.2.0、相同來源與編譯參數，計算前也都設定為 FE_TONEAREST。WSL 使用本地建置的 GCC，Mac 則使用 Homebrew 版本，因此是相同 release 的跨平台編譯器，而不是同一份 compiler binary。

兩端都以 -O2 編譯，並關閉 fast-math、FMA 與 LTO，同時保留 -frounding-math。為避免中間結果保留在額外精度的暫存狀態或被重新組合，程式利用 GCC inline assembly 的 memory constraint，要求 sum 先依平台的 long double 格式存回記憶體，再執行減法；輸入為 $a=2^{53}$ 、 $b=1$ 。

```
long double sum = a + b;
asm volatile("" : "+m"(sum) : : "memory");
return sum - a;
```

圖 5-1 將中間結果依平台 long double 格式存回記憶體後再進行減法。

```
alan@TUF-F16:/mnt/c/Users/ALAN/Desktop/floating-point-hw2$ for f in nearest upward downward towardzero; do
echo "=== $f ==="
grep -E '^^(compiler=|HW2_ROUNDING=|long_double_bytes=|double_half_ulp_at_one=|abi_case_rounding=|long_double_big=|long_double_increment=)' \
"results/Linux-20261004-201349/q5-$f.txt"
done
=== nearest ===
compiler=16.2.0
HW2_ROUNDING=nearest
long_double_bytes=16,digits=64,max_exponent=16384
double_half_ulp_at_one=1,hex=0x1p+0
abi_case_rounding=FE_TONEAREST
long_double_big=9007199254740992,hex=0x8p+50
long_double_increment=1,hex=0x8p-3
=== upward ===
```

圖 5-2 WSL：long double 有 64 個有效位，增量計算得到 1。

觀察	Windows／WSL	Apple M4 Mac
儲存空間／有效位數	16 bytes／64 bits	8 bytes／53 bits
max_exponent	16384	1024
捨入方式	FE_TONEAREST	FE_TONEAREST
$(2^{53}+1)-2^{53}$	1，hex 0x8p-3	0，hex 0x0p+0

```

alan@Mac Floating-Point % for f in nearest upward downward towardzero; do
echo "=== $f ==="
grep -E '^(\bcompiler=|HW2_ROUNDING=|long_double_bytes=|double_half_ulp_at_one=|abi_case_rounding=|long_double_big=|long_double_increment=)' \
"$MAC_RESULTS/q5-$f.txt"
done
=== nearest ===
compiler=16.2.0
HW2_ROUNDING=nearest
long_double_bytes=8,digits=53,max_exponent=1024
double_half_ulp_at_one=1,hex=0x1p+0
abi_case_rounding=FE_TONEAREST
long_double_big=9007199254740992,hex=0x1p+53
long_double_increment=0,hex=0x0p+0

```

圖 5-3 Apple M4 Mac：long double 有 53 個有效位，相同增量計算得到 0。

兩端的 assembly 都保留了加法、存回與減法：WSL 為 FADD／FSTPT／FLDT／FSUBP，Mac 為 FADD／STR／LDR／FSUB。輸入皆為有限值，也沒有溢位、越界或未初始化讀取，因此不同答案不是由刪除運算或未定義行為造成。

差異來自 long double 的實際精度。WSL 有 64 個有效位，能精確保留 $2^{53}+1$ ，因此減回 2^{53} 後得到 1；Mac 只有 53 個有效位，在 2^{53} 附近相鄰可表示值的間隔為 2，+1 恰好位於中點，在最接近值捨入下回到 2^{53} ，最後得到 0。這與 Apple arm64 ABI 中 long double 與 double 具有相同精度的規格吻合。[6] 型態參數、實際輸出與 assembly 一起支持這個跨平台差異。

Q6. 總結

經過前面的實驗，我認為浮點數並不是不能使用，而是不能把它直接視為數學上的實數。它只有有限精度，而且實際行為還會受到硬體指令、編譯器最佳化、函式庫與平台 ABI 影響。同一段看似簡單的計算，換一種型態、改變運算順序、開啟不同最佳化，甚至只換到另一個平台，都可能在精度或速度上出現差異。

Q1 的 benchmark 顯示，型態大小並不能直接決定運算速度，而有限有效位數也會讓大數附近的小增量消失；Q2 從 assembly 看到 `std::sqrt` 在本機會使用特定的硬體平方根指令；Q3 則證明 `-ffast-math` 可以透過重新結合運算及對特殊值的假設直接改變答案。Q4 顯示輸出到固定小數位仍要考慮捨入規則，Q5 更進一步說明，即使程式碼、編譯器 release 與參數相同，不同 ABI 對 long double 的定義仍可能讓結果不同。

因此，在科學計算或遊戲物理中使用浮點數時，我會先確認問題能接受多少誤差，再選擇合適的型態與計算方式，而不是單純認為 double 一定比 float 安全，或在一台機器得到合理結果就假設所有環境都會相同。對結果敏感的部分還需要固定重要的編譯條件，並在實際目標平台上測試。

這種誤差也會影響程式中的相等判斷。許多十進位小數無法用有限的二進位浮點數精確表示，因此兩個在數學上應該相同的計算結果，經過不同運算與捨入後仍可能留下微小差異。例如以 double 計算 $0.1 + 0.2$ 時，就不應假設它一定能用 `==` 與 0.3 做 bit-for-bit 的相等判斷。對科學計算中的收斂條件、遊戲中的位置或碰撞判定而言，直接依賴完全相等都可能造成不符合預期的結果。

```
double x = 0.1 + 0.2;
double y = 0.3;

// 對經過計算的浮點值直接要求完全相等，可能過於嚴格。
bool same = (x == y);
```

圖 6-1 浮點數相等比較的常見陷阱。

實務上應依問題本身的尺度與精度需求定義容許範圍，例如使用 **absolute tolerance** 比較 $|a-b|$ ，數值尺度變化較大時再搭配 **relative tolerance**。容許值也不應直接套用一個固定的「萬用 **epsilon**」。整份實驗最後得到的重點不是避免使用浮點數，而是清楚知道誤差可能從哪裡產生、哪些假設會受到平台與編譯設定影響，並用符合實際需求的方式驗證結果。

Q7. AI 使用自我揭露（不記入作業分數）

我：建置實驗環境、進程式碼 **review**、執行各項實驗與 **Benchmark**、在 WSL 與 Mac 上進行測試、觀察與比較實驗結果，並查找相關資料與規格，整理對浮點數運算的理解與結論。

AI：實作與修改實驗程式碼、設計部分測試案例、查找相關資料、解讀 **assembly**，以及整理報告內容與排版。

參考資料

規格與演算法說明參考下列資料；本報告中的執行行為則以實際輸出與編譯後指令進行核對。

- [1] C++ working draft c.math
<https://eel.is/c++draft/c.math>
- [2] GCC 16.2.0 Optimize Options
<https://gcc.gnu.org/onlinedocs/gcc-16.2.0/gcc/Optimize-Options.html>
- [3] glibc Floating Point Parameters
https://sourceware.org/glibc/manual/latest/html_node/Floating-Point-Parameters.html
- [4] Intel SDM 官方入口
<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- [5] glibc Exponents and Logarithms
https://sourceware.org/glibc/manual/latest/html_node/Exponents-and-Logarithms.html
- [5a] glibc generic double e_sqrt.c
https://raw.githubusercontent.com/bminor/glibc/master/sysdeps/ieee754/dbl-64/e_sqrt.c
- [5b] glibc Rounding Modes
https://sourceware.org/glibc/manual/latest/html_node/Rounding.html
- [6] Apple Writing ARM64 code
<https://developer.apple.com/documentation/xcode/writing-arm64-code-for-apple-platforms>